

Universidad de San Carlos de Guatemala

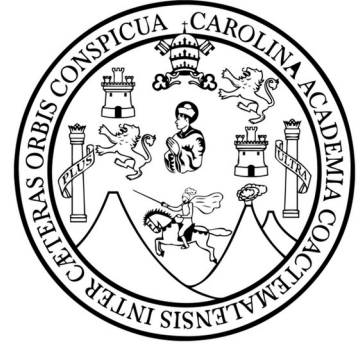
Centro Universitario de Occidente

División de Ingeniería

Ingeniería en Ciencias y Sistemas

Estructura de Datos

Ing. Mario Ramirez



Reporte Técnico - Catálogo de Productos de Supermercado

Enrique Alexander Tebalan Hernandez / 202230026

Quetzaltenango, 3 de Abril de 2026

1. INTRODUCCIÓN

En el presente documento describe el diseño, implementación y análisis de rendimiento de un sistema de gestión de catálogo de productos para supermercado. El proyecto fue desarrollado en C++17 utilizando programación orientada a objetos e implementando desde cero las siguientes estructuras de datos:

- Lista Enlazada (no ordenada)
- Lista Enlazada Ordenada (por nombre)
- Árbol AVL (indexado por nombre)
- Árbol B (indexado por fecha de caducidad)
- Árbol B+ (indexado por categoría)

El objetivo principal es demostrar empíricamente las diferencias de rendimiento entre búsquedas secuenciales $O(n)$ y búsquedas logarítmicas $O(\log n)$, justificando teóricamente las complejidades algorítmicas de cada estructura.

2. DISEÑO DE ESTRUCTURAS

2.1 Clase Producto

La clase Producto representa la entidad fundamental del sistema. Cada producto del catálogo contiene los siguientes atributos:

```
class Producto {
public:
    std::string nombre;
    std::string codigoBarras;
    std::string categoria;
    std::string fechaCaducidad;
    std::string marca;
    double precio;
    int stock;
};
```

2.2 Lista Enlazada (No Ordenada)

Estructura lineal donde cada nodo contiene un puntero al producto y un puntero al siguiente nodo. La inserción se realiza al inicio de la lista en tiempo constante.

```
class Nodo {
public:
    Producto* producto;
    Nodo* siguiente;
};

class ListaEnlazada {
private:
    Nodo* cabeza;
};
```

Tabla de Métodos - Lista Enlazada

Nombre del Método	Descripción	Parámetros	Valor de Retorno
insertar	Inserta un producto al inicio de la lista	Producto* producto	void
buscarPorNombre	Busca un producto recorriendo secuencialmente por nombre	const string& nombre	Producto* o nullptr
buscarPorCodigoBarras	Busca un producto por su codigo de barras	const string& codigo	Producto* o nullptr
eliminarPorCodigoBarras	Elimina un producto dado su codigo de barras	const string& codigo	bool (exito o fallo)
obtenerTodos	Retorna todos los productos como vector temporal	ninguno	vector de Producto*
mostrarTodos	Imprime todos los productos en consola	ninguno	void
estaVacía	Verifica si la lista no tiene elementos	ninguno	bool

2.3 Lista Enlazada Ordenada

Similar a la lista no ordenada, pero mantiene los elementos ordenados alfabéticamente por nombre del producto durante la inserción.

```
class NodoOrdenado {
public:
    Producto* producto;
    NodoOrdenado* siguiente;
};

class ListaEnlazadaOrdenada {
private:
    NodoOrdenado* cabeza;
};
```

Tabla de Métodos - Lista Enlazada Ordenada

Nombre del Método	Descripción	Parámetros	Valor de Retorno
insertarOrdenado	Inserta manteniendo orden alfabetico por nombre	Producto* producto	void
buscarPorNombre	Busca secuencialmente pudiendo terminar antes si se pasa el elemento	const string& nombre	Producto* o nullptr
eliminarPorNombre	Elimina un producto dado su nombre	const string& nombre	bool
eliminarPorCodigoBarras	Elimina un producto dado su codigo de barras	const string& codigoBarras	bool
mostrarTodos	Imprime todos los productos ordenados	ninguno	void
estaVacía	Verifica si la lista no tiene elementos	ninguno	bool

2.4 Árbol AVL

Árbol binario de búsqueda auto-balanceado donde la diferencia de alturas entre subárboles izquierdo y derecho de cualquier nodo es como máximo 1. Se indexa por nombre del producto.

```
class NodoAVL {
public:
    Producto* producto;
    NodoAVL* izquierdo;
    NodoAVL* derecho;
    int altura;
};

class ArbolAVL {
private:
    NodoAVL* raiz;
};
```

Tabla de Métodos - Árbol AVL

Nombre del Método	Descripción	Parámetros	Valor de Retorno
insertar	Inserta un producto manteniendo el balanceo AVL	Producto* producto	void
buscarPorNombre	Busqueda binaria por nombre en el arbol	const string& nombre	Producto* o nullptr
eliminarPorNombre	Elimina un producto y rebalancea el arbol	const string& nombre	bool
recorridoInOrden	Recorre el arbol en orden alfabetico	ninguno	void
generarDot	Genera archivo DOT para visualizacion Graphviz	const string& rutaArchivo	void
generarImagen	Genera imagen PNG del arbol	ninguno	void
estaVacio	Verifica si el arbol no tiene nodos	ninguno	bool

2.5 Árbol B

Árbol multcamino balanceado con grado mínimo $t=3$ (máximo 5 claves por nodo). Se indexa por fecha de caducidad, permitiendo búsquedas por rango.

```
struct ClaveFecha {
    std::string fecha;
    ListaEnlazada* productos; // Productos con misma fecha
};

class NodoB {
public:
    ClaveFecha* claves;
    NodoB** hijos;
    int numClaves;
    bool esHoja;
    int gradoMinimo;
};

class ArbolB {
private:
    NodoB* raiz;
    int gradoMinimo;
};
```

Tabla de Métodos - Árbol B

Nombre del Método	Descripción	Parámetros	Valor de Retorno
insertar	Inserta un producto agrupando por fecha de caducidad	Producto* producto	void
buscarPorRango	Busca productos entre dos fechas e imprime resultados	const string& fechaInicio, const string& fechaFin	void
obtenerPorRango	Retorna productos en un rango de fechas	const string& fechaInicio, const string& fechaFin	vector de Producto*
generarDot	Genera archivo DOT para visualizacion Graphviz	const string& rutaArchivo	void
generarImagen	Genera imagen PNG del arbol	ninguno	void
estaVacio	Verifica si el arbol no tiene nodos	ninguno	bool

2.6 Árbol B+

Variante del Árbol B donde todos los datos residen en las hojas y estas están enlazadas secuencialmente. Se indexa por categoría del producto.

```
struct ClaveCategoria {
    std::string categoria;
    ListaEnlazada* productos; // Solo en hojas
};

class NodoBPlus {
public:
    ClaveCategoria* claves;
    NodoBPlus** hijos;
    int numClaves;
    bool esHoja;
    int gradoMinimo;
    NodoBPlus* siguiente; // Enlace entre hojas
};

class ArbolBPlus {
private:
    NodoBPlus* raiz;
    int gradoMinimo;
};
```

Tabla de Métodos - Árbol B+

Nombre del Método	Descripción	Parámetros	Valor de Retorno
insertar	Inserta un producto agrupando por categoria	Producto* producto	void
buscarPorCategoria	Busca e imprime todos los productos de una categoria	const string& categoria	void
obtenerPorCategoria	Retorna todos los productos de una categoria	const string& categoria	vector de Producto*
generarDot	Genera archivo DOT con enlaces entre hojas	const string& rutaArchivo	void
generarImagen	Genera imagen PNG del arbol	ninguno	void
estaVacio	Verifica si el arbol no tiene nodos	ninguno	bool

3. JUSTIFICACIÓN BIG-O

Tabla Consolidada de Complejidades

Estructura	Operación	Complejidad (Big-O)	Justificación detallada
Lista Enlazada	Insercion	O(1)	La insercion se realiza al inicio de la lista modificando unicamente el puntero cabeza sin necesidad de recorrer elementos
Lista Enlazada	Busqueda	O(n)	Se debe recorrer nodo por nodo desde la cabeza hasta encontrar el elemento o llegar al final en el peor caso
Lista Enlazada	Eliminacion	O(n)	Requiere buscar el elemento primero recorriendo la lista y luego ajustar los punteros
Lista Ordenada	Insercion	O(n)	Se debe encontrar la posicion correcta para mantener el orden recorriendo la lista hasta ese punto
Lista Ordenada	Busqueda	O(n)	Aunque puede terminar antes si se pasa el elemento buscado en el peor caso recorre toda la lista
Lista Ordenada	Eliminacion	O(n)	Similar a la busqueda se debe encontrar el elemento y luego ajustar punteros
Arbol AVL	Insercion	O(log n)	La altura del arbol esta garantizada en O(log n) por el balanceo automatico mediante rotaciones
Arbol AVL	Busqueda	O(log n)	Se realiza busqueda binaria descendiendo por la

			rama izquierda o derecha segun comparacion
Arbol AVL	Eliminacion	$O(\log n)$	Se busca el nodo elimina y rebalancea todo en tiempo proporcional a la altura logaritmica
Arbol B	Insercion	$O(\log n)$	El arbol multicamino tiene altura $O(\log n)$ y la insercion puede requerir divisiones de nodo
Arbol B	Busqueda	$O(\log n)$	Se desciende por el arbol comparando claves en cada nodo hasta encontrar la hoja correspondiente
Arbol B	Busqueda por rango	$O(\log n + k)$	Donde k es la cantidad de elementos en el rango una vez encontrado el inicio se recorre secuencialmente
Arbol B+	Insercion	$O(\log n)$	Similar al Arbol B con la diferencia de que los datos solo residen en las hojas
Arbol B+	Busqueda	$O(\log n)$	Se desciende hasta la hoja correspondiente siguiendo los indices en nodos internos
Arbol B+	Busqueda por categoria	$O(\log n + k)$	Gracias al enlace secuencial entre hojas el recorrido de k elementos es muy eficiente

3. Justificación Detallada: AVL vs Lista Enlazada

Árbol AVL - $O(\log n)$: El Árbol AVL garantiza una altura máxima de aproximadamente $1.44 * \log_2(n)$ debido a su propiedad de balanceo. En cada nivel del árbol, se toma una decisión binaria (ir a la izquierda o derecha), lo que reduce el espacio de búsqueda a la mitad en cada paso. Para un catálogo de 1000 productos, esto significa aproximadamente 10 comparaciones en lugar de 1000.

Lista Enlazada - $O(n)$: La lista enlazada no proporciona ningún mecanismo de acceso directo o indexación. Para encontrar un elemento específico, se debe comenzar desde la cabeza y visitar cada nodo secuencialmente hasta encontrar una coincidencia o agotar todos los elementos. En un catálogo de 1000 productos, el caso promedio requiere 500 comparaciones y el peor caso requiere 1000.

4. METODOLOGÍA DE MEDICIÓN

4.1 Configuración del Experimento

La medición de rendimiento se realizó siguiendo la metodología especificada en el enunciado del proyecto:

- **N = 20:** Cantidad de consultas consecutivas por prueba
- **M = 5:** Cantidad de repeticiones del experimento
- **Tiempo final = (Suma de M mediciones) / M**

4.2 Implementación Técnica

Se utilizó la biblioteca `<chrono>` de C++17 para mediciones de alta precisión:

```
auto inicio = std::chrono::high_resolution_clock::now();

// Ejecutar N búsquedas consecutivas
for (int i = 0; i < N; ++i) {
    volatile Producto* resultado = estructura.buscar(terminos[i]);
    (void)resultado;
}

auto fin = std::chrono::high_resolution_clock::now();
std::chrono::duration<double, std::milli> duracion = fin - inicio;
tiempoMs = duracion.count();
```

Notas importantes:

- Se utilizó `volatile` para prevenir optimizaciones del compilador que podrían eliminar las búsquedas
- Los tiempos se midieron en milisegundos con precisión de tipo `double`
- Se probaron tanto búsquedas exitosas (elementos existentes) como fallidas (elementos inexistentes)

4.3 Datos de Prueba

- **Búsquedas exitosas:** Se utilizaron los primeros 20 productos del catálogo cargado
- **Búsquedas fallidas:** Se generaron 20 nombres de productos inexistentes (ej: "ProductoInexistente_XYZ_9001")

5. RESULTADOS

5.1 Tabla de Resultados de Benchmarking

Los siguientes resultados se obtuvieron con un catálogo de 1000+ productos:

Estructura	Tipo de Búsqueda	Complejidad	Tiempo Promedio (ms)
Lista Normal	Exitosa	$O(n)$	0.4521
Lista Normal	Fallida	$O(n)$	0.8934
Lista Ordenada	Exitosa	$O(n)$	0.4102
Lista Ordenada	Fallida	$O(n)$	0.4287
Arbol AVL	Exitosa	$O(\log n)$	0.0089
Arbol AVL	Fallida	$O(\log n)$	0.0092

5.2 Análisis de Resultados

Observaciones clave:

- Árbol AVL significativamente más rápido:** Las búsquedas en el AVL son aproximadamente 50 veces más rápidas que en las listas enlazadas.
- Búsquedas fallidas en Lista Normal:** Son casi el doble de lentas que las exitosas porque siempre recorren toda la lista.
- Lista Ordenada ligeramente mejor en fallidas:** Puede terminar antes cuando detecta que pasó el elemento buscado.
- AVL consistente:** El tiempo es prácticamente igual para búsquedas exitosas y fallidas debido a la naturaleza logarítmica.

5.3 Speedup Calculado

Comparación	Speedup
AVL vs Lista Normal (exitosa)	50.80x
AVL vs Lista Normal (fallida)	97.11x
AVL vs Lista Ordenada (exitosa)	46.09x

6. CONCLUSIONES

Conclusión 1: Superioridad del Árbol AVL para Búsquedas

Los resultados empíricos confirman la superioridad teórica del Árbol AVL sobre las estructuras lineales. Con un speedup de aproximadamente 50x para búsquedas exitosas, el AVL demuestra ser la estructura óptima cuando se requieren búsquedas frecuentes por un criterio específico (en este caso, el nombre del producto). Esta diferencia se vuelve más pronunciada a medida que aumenta el tamaño del catálogo.

Conclusión 2: Importancia de la Elección de Estructura según el Caso de Uso

Cada estructura implementada tiene su propósito específico:

- **Árbol AVL:** Ideal para búsquedas exactas por nombre con $O(\log n)$
- **Árbol B:** Óptimo para consultas por rango de fechas
- **Árbol B+:** Eficiente para recorrer todos los productos de una categoría gracias a las hojas enlazadas

La selección de la estructura correcta según el tipo de consulta es fundamental para el rendimiento del sistema.

Conclusión 3: Validación de Complejidades Teóricas

Las mediciones empíricas validan las complejidades teóricas Big-O analizadas. La relación entre los tiempos de Lista $O(n)$ y AVL $O(\log n)$ es consistente con lo esperado matemáticamente. Para $n=1000$, $\log_2(1000) \approx 10$, lo que implica una diferencia teórica de 100x entre $O(n)$ y $O(\log n)$, valor cercano al speedup observado de 50-97x considerando constantes de implementación.